ATIVIDADE ACADÊMICA · ENGENHARIA DE SOFTWARE

Manual de Gestão de Código e Entrega de Software

Sistema DKRT: Auditor de Conformidade de Entidades de Contexto

Branches, ambientes, revisão, CI/CD, hotfix, feature flags e organização das aplicações

Grupo: [preencher com os nomes dos integrantes]

Repositório: TCC-CEFSA-DKRT/TCC_DKRT (GitHub)

Versão 1.3 · outubro de 2026

Sumário

Apresentação e contexto..... 3
1. Estratégia de versionamento..... 5
2. Processo de desenvolvimento..... 7
3. Integração Contínua (CI)..... 9
4. Entrega e implantação (CD)..... 11
5. Tratamento de bugs em produção..... 13
6. Funcionalidades ainda não concluídas..... 14
7. Arquitetura da solução e pipelines..... 15
8. Decisões e justificativas..... 16
Referências..... 17

Como ler: as seções 1 a 7 respondem, na ordem, às sete perguntas do enunciado. A seção 8 reúne as decisões e as justificativas, com a alternativa descartada e o custo aceito em cada uma.

Figuras: Figura 1 (workflow Git, seção 1) e Figura 2 (pipeline de CI/CD e hotfix, seção 3).

Apresentação e contexto

Este manual define como o grupo gerencia o código-fonte do sistema DKRT e como cada alteração chega aos usuários. A regra de ouro é simples: **a branch `main` é a produção**. Nada entra nela sem antes ser testado pelo Q.A em um ambiente de desenvolvimento (Dev) e em um de homologação, e só o Tech Lead autoriza o último passo.

O sistema em uma linha

O DKRT audita entidades de um Context Broker (Orion, padrão NGSI-v2) contra os Smart Data Models, em dois níveis: validação estrutural e heurística determinística (Nível 1) e análise semântica por um SLM local com RAG (Nível 2). O código é um monolito modular em arquitetura hexagonal (`domain`, `application`, `infrastructure`, `interfaces`), com um frontend Angular separado.

O que é entregue e onde roda

Componente	Tecnologia	Produção (ADR-0017)	Self-hosted (ADR-0019)
API + Worker	Python 3.11, FastAPI; um único pacote `src/auditor`	3 funções AWS Lambda (api, worker, worker-dlq) de uma mesma imagem de contêiner no ECR, atrás de um ALB	Um processo uvicorn (API + worker) em contêiner (`selfhosted/Dockerfile`)
Frontend	Angular 19	Build estático em S3 + CloudFront	Build estático servido por nginx (`selfhosted/frontend.Dockerfile`)
Banco, fila e RAG	PostgreSQL + pgvector; migrations Alembic; SQS	RDS e SQS	Postgres em contêiner; fila em memória
SLM	Ollama (Qwen2.5)	Instância EC2 com GPU (ADR-0013/0015)	Contêiner Ollama
Infraestrutura	Terraform (`infra/terraform`)	Ambientes `homolog` e `production`	Não se aplica
dataspace-demo	Stack didática com docker compose próprio	Não é implantada	Não se aplica; fornece os 3 cenários de teste de aceitação

Premissas do grupo

- Equipe pequena de estudantes trabalhando em paralelo no mesmo repositório (privado, organização GitHub do projeto), com três papéis: **Dev** (quem desenvolve; neste manual, "o Dev" é o desenvolvedor, e "DEV" em maiúsculas é o ambiente), **Q.A** (quem testa e aprova) e **Tech Lead** (quem revisa, cria a release e autoriza produção).
- Um produto, uma linha de produção e **quatro ambientes**: local, **DEV**, **homologação** e **produção**. O modo self-hosted (ADR-0019) é um canal de distribuição, não um quinto ambiente do time.
- A qualidade é garantida em **dois ciclos de teste do Q.A**: o primeiro no DEV, sobre a própria branch do desenvolvedor, e o segundo na homologação, sobre a release candidata.
- Processo proporcional ao porte: rigoroso o bastante para que a produção nunca receba código não aprovado, leve o bastante para ser seguido por todos.

Resumo das decisões

Pergunta do enunciado	Decisão do grupo
1. Versionamento	Fluxo em funil inspirado no Git Flow, **sem `develop`**: `main` = produção; `release/X.Y.Z` = homologação. Branches de trabalho (`feature/`, `improve/`, `bug/`) **nascem da `main`**. O DEV é um deploy sob demanda da própria branch. Versões marcadas por tags: `vX.Y.Z-rc.N` (homologação) e `vX.Y.Z` (produção).
2. Processo de desenvolvimento	Branch da `main` → teste local → PR para a `main` (URL ao Tech Lead) → deploy da branch no DEV → Q.A (1° ciclo) → Tech Lead cria a release → tag `-rc` → Q.A na homologação (2° ciclo) → merge da release na `main` → tag final → produção.
3. CI	A cada PR para a `main` e a cada push em `release/*`: testes do backend e do frontend, qualidade, segurança, validação de Terraform e build das imagens. Nenhum merge sem tudo verde.
4. CD	O DEV recebe a branch por **deploy manual sob demanda**. Homologação e produção implantam por **tag**: `vX.Y.Z-rc.N` publicada na release sobe a homologação; o merge na `main` seguido da tag `vX.Y.Z` e da **aprovação manual do Tech Lead** sobe a produção, promovendo o mesmo artefato testado.
5. Bug em produção	Branch `bug/` criada a partir da `main`, com **atalho**: não passa pelo DEV, vai direto para release de correção (PATCH), homologação focada e produção.
6. Funcionalidade inacabada	A branch de trabalho **é** a unidade de entrega: só vai para release quando pronta e aprovada. Feature flag (`FEATURE_*`) é a exceção, para o que precisa entrar em produção em partes.
7. Arquitetura	Monorepo com um só fluxo de branches, **pipelines independentes por pasta** e componentes implantáveis separadamente (frontend, API/worker e infraestrutura).

1. Estratégia de versionamento

1.1 Qual workflow Git foi adotado

O grupo adota um **fluxo em funil**, inspirado no Git Flow, com duas diferenças importantes. As branches de trabalho **nascem da `main`**, isto é, da produção, e não de uma `develop`. E **não existe branch `develop`**: o ambiente DEV recebe a própria branch de trabalho por deploy sob demanda. Por isso, o que o Q.A testa no DEV é exatamente o que chega à release, sem o trabalho em andamento de outros integrantes.

Só uma branch é permanente, a `main` (produção). A homologação usa uma branch temporária, a `release/X.Y.Z`, criada pelo Tech Lead a partir da branch de trabalho aprovada no DEV. Não há branch permanente de homologação: a subida é feita por **tags** publicadas na release (`vX.Y.Z-rc.N`) e, depois do merge, na `main` (`vX.Y.Z`). A tag é um marcador imutável de um commit; é ela que dispara o deploy e que identifica exatamente o que foi testado.

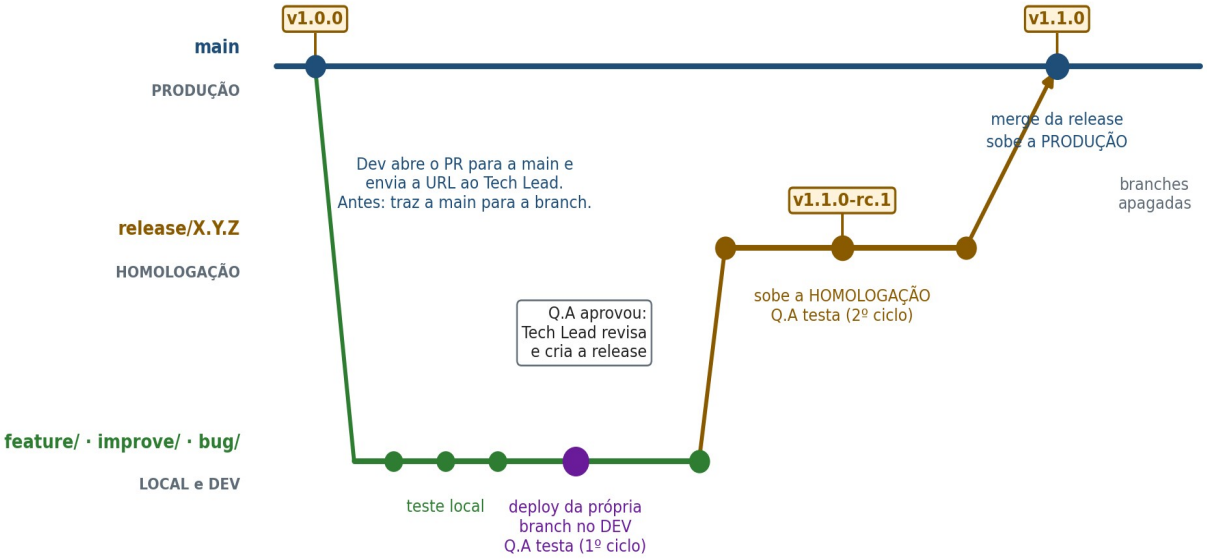


Figura 1: Workflow Git adotado. Cada linha é uma branch; o rótulo abaixo do nome indica o ambiente. As tags -rc sobem a homologação e a vX.Y.Z sobe a produção.

1.2 Branches e finalidades

Branch	Finalidade	Origem	Vida útil e encerramento
`main`	Produção. Só recebe releases aprovadas pelo Q.A na homologação. Cada release recebe uma tag.	Não se aplica	Permanente. Protegida: sem push direto, sem force push, sem exclusão. Só o Tech Lead faz merge.

`feature/<tema>`
`improve/<tema>`
`bug/<tema>`
	Branch de trabalho: nova funcionalidade, melhoria (inclui refatoração e documentação) ou correção de bug. É implantada no DEV, sob demanda, para o primeiro ciclo do Q.A.	`main` atualizada	Até a release ser mergeada na `main`; então é apagada.

Branch	Finalidade	Origem	Vida útil e encerramento
`release/X.Y.Z` ex.: **`release/1.2.0`**	Candidata a produção. A tag `vX.Y.Z-rc.N` publicada nela implanta na homologação; é o que o Q.A testa e o que será mergeado na `main`.	A branch de trabalho aprovada no DEV, atualizada com a `main`	Criada pelo Tech Lead. Apagada após o merge na `main`; a tag permanece.
Tags `vX.Y.Z-rc.N` e `vX.Y.Z`	É a tag que dispara a implantação. A `-rc.N`, publicada na release, sobe a homologação; a `vX.Y.Z`, publicada na `main` após o merge, sobe a produção.	Release (`-rc.N`) ou `main` (final)	Imutáveis. Nunca são movidas nem apagadas. Cada correção na homologação gera um novo `-rc`.

1.3 Regras que mantêm o fluxo coerente

O fluxo só funciona se estas quatro regras forem seguidas sempre:

1. **A branch de trabalho sempre nasce da `main`** e traz a `main` para dentro dela (`git merge origin/main`) antes do PR e antes da release. Só assim os conflitos aparecem cedo e a release leva apenas a alteração dela sobre a produção.
2. **O DEV recebe uma branch por vez.** Quem precisa do DEV avisa o grupo; o deploy é manual (GitHub Actions, escolhendo a branch) e protegido por fila (*concurrency*), para que dois deploys não se atropelem.
3. **Correções pedidas pelo Q.A nascem na própria branch de trabalho.** Depois o Dev implanta de novo no DEV; se o problema surgiu na homologação, o Tech Lead atualiza a release e publica uma nova tag `-rc`.
4. **A `main` só recebe releases** aprovadas na homologação. Ninguém faz push direto nela.

1.4 Quando criar e quando encerrar uma branch

- **Criar** a branch de trabalho a partir da `main` recém-atualizada, com um objetivo pequeno e verificável, de preferência registrado numa issue. O prefixo indica o tipo: `feature/` (nova funcionalidade), `improve/` (melhoria, refatoração, documentação) e `bug/` (correção).
- **Encerrar** todas as branches da alteração (a de trabalho e a `release/X.Y.Z`) quando a release for mergeada na `main`. O GitHub apaga a branch automaticamente após o merge de PR (opção *Automatically delete head branches*); a tag mantém o registro.
- **Branch parada:** sem commits há 7 dias, a branch é revisada em reunião do grupo: retomada, fatiada ou descartada. Como a branch de trabalho sai da `main`, ela precisa trazer a `main` para dentro dela com frequência para não acumular conflitos.
- **Mensagens de commit** seguem Conventional Commits (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`), o que alimenta as notas de release.
- **Versionamento semântico** (`vMAJOR.MINOR.PATCH`): MAJOR para quebra de contrato (API REST ou migração incompatível), MINOR para funcionalidade ou melhoria compatível, PATCH para correção de bug. O nome da release acompanha a versão (`release/1.2.0`) e as tags de homologação ficam `v1.2.0-rc.1`, `v1.2.0-rc.2` e assim por diante. API e frontend compartilham a mesma versão porque são liberados juntos. A primeira release será `1.0.0`, correspondente à entrega para a banca.

2. Processo de desenvolvimento

2.1 Como uma funcionalidade é iniciada e integrada

1. **Issue.** A alteração é registrada no GitHub Issues com objetivo e critério de aceite.
2. **Branch de trabalho.** O Dev cria a branch a partir da `main` atualizada: `git switch main && git pull --ff-only && git switch -c feature/login`.
3. **Desenvolvimento e teste local.** Ciclos pequenos, com teste escrito junto do código. Antes de seguir, o Dev roda `pytest tests/unit`, `ng test` e o ambiente local (`docker compose up -d`, `uvicorn`, `ng serve`); para mudanças na auditoria, também os cenários do `dataspace-demo`.
4. **Atualizar com a `main` e abrir o PR.** O Dev traz a `main` para dentro da branch (`git merge origin/main`), resolve os conflitos ali e abre o PR da branch para a `main`, que roda o CI. O PR é o lugar da conversa e da revisão; a URL é enviada ao Tech Lead.
5. **Primeiro ciclo de testes (DEV).** Com o CI verde, o Dev dispara o deploy da própria branch no ambiente DEV (GitHub Actions, escolhendo a branch) e o Q.A faz os primeiros testes. Se reprovar, o Dev corrige na mesma branch e implanta de novo.
6. **Revisão e release.** Com a aprovação do Q.A no DEV, o Tech Lead revisa o código do PR e cria `release/X.Y.Z` **a partir da branch de trabalho**. A release é uma cópia congelada do que foi aprovado.
7. **Segundo ciclo de testes (homologação).** O Tech Lead publica a tag `vX.Y.Z-rc.1` na release. A tag dispara o deploy na homologação, onde o Q.A testa a candidata. Se reprovar, a correção entra na branch de trabalho, o Tech Lead atualiza a release e publica `-rc.2`, repetindo o ciclo.
8. **Produção.** Com a aprovação do Q.A, o Tech Lead faz o merge da `release/X.Y.Z` na `main` (o PR da branch fecha sozinho), publica a tag `vX.Y.Z` (que dispara o deploy em produção) e aprova a implantação (seção 4).
9. **Encerramento.** As branches de trabalho e `release/X.Y.Z` são apagadas e a issue é fechada. A tag permanece como registro da versão.

2.2 Pull Request, aprovação e verificações

Há um único PR por alteração, da branch de trabalho para a `main`, e duas etapas de aprovação com rigor crescente:

Pergunta	Primeiro ciclo (DEV)	Release e produção (`main`)
Haverá PR?	Sim, da branch de trabalho para a `main`, aberto antes do deploy no DEV. Não há push direto na `main`.	O mesmo PR. O merge final é da `release/X.Y.Z`, e o PR fecha sozinho.
Quem aprova?	O **Q.A** aprova o comportamento no DEV (registra no PR). Um integrante diferente do autor revisa o código, se o Tech Lead delegar.	O **Tech Lead** revisa o código. O Q.A registra no PR a aprovação da homologação.
Quem faz o merge e o deploy?	O **Dev** dispara o deploy no DEV. Ninguém faz merge ainda.	Somente o **Tech Lead** (ou o substituto designado) cria a release, publica as tags e faz o merge, depois da aprovação do Q.A na homologação.

Pergunta	Primeiro ciclo (DEV)	Release e produção (`main`)
O que é verificado?	1) CI verde (seção 3); 2) branch atualizada com a `main` e sem conflitos; 3) testes locais executados (checklist do PR).	1) CI verde na release; 2) release atualizada com a `main`; 3) Q.A aprovou no DEV **e** na homologação; 4) migrações e `dataspace-demo` verificados (seção 3.2); 5) ADR em `docs/adr/` se houver decisão arquitetural.

2.3 Por que este desenho

- **O Q.A testa o que vai para a produção:** como o DEV recebe a própria branch, e não um ambiente que mistura o trabalho de vários Devs, o que o Q.A aprova no primeiro ciclo é exatamente o que vira release. Não há surpresa de código alheio.
- **Dois ciclos de Q.A:** o primeiro, no DEV, é rápido e barato e detecta erro cedo; o segundo, na homologação, valida a release candidata, já atualizada com a `main`, o que também é o momento em que alterações de colegas se encontram.
- **Tech Lead como porteiro da `main`:** com vários integrantes alterando partes diferentes do sistema, um único responsável por revisar, criar a release e mergear garante que nada chegue à produção sem revisão e sem a aprovação do Q.A, e dá a quem responde pela entrega o controle do que entra.

2.4 Aplicação das regras na plataforma

As regras são impostas pela própria plataforma, e não apenas combinadas: *ruleset* na `main` e em `release/*` (PR obrigatório, sem push direto, sem force push, sem exclusão, e somente o Tech Lead na lista de bypass); um arquivo `CODEOWNERS` que torna o Tech Lead revisor obrigatório de `infra/`, `migrations/` e `docs/adr/`; e o *Environment* `producao` do GitHub com o Tech Lead como único aprovador de deploy. Um template de PR traz o checklist de testes locais, atualização com a `main`, migração, flag e ADR.

3. Integração Contínua (CI)

3.1 O que acontece automaticamente após a submissão

O pipeline roda em **todo Pull Request** para a `main`, em todo push em `release/*` e antes de cada deploy no DEV. Além disso, uma execução **agendada semanal** repete as verificações de segurança (novas vulnerabilidades surgem mesmo sem commits) e os testes *golden* do SLM. Cada pasta do monorepo tem seu workflow, acionado por filtro de caminho (seção 7), de modo que uma alteração só no frontend não executa os testes pesados do backend.

Etapa	O que executa	Bloqueia o merge?
Testes do backend	`pytest tests/unit` com cobertura. Em seguida, job de integração com PostgreSQL/pgvector e Orion como serviços, rodando `alembic upgrade head` em banco vazio, `alembic heads` (deve haver uma só) e `pytest -m integration`.	Sim
Testes do frontend	`npm ci`, `ng test` em Chrome headless e `ng build --configuration production`.	Sim
Qualidade de código	`ruff check` e `mypy` sobre o código alterado.	Sim
Segurança	`pip-audit` e `npm audit --omit=dev` (vulnerabilidades críticas e altas), varredura de segredos e Dependabot.	Sim (críticas e altas)
Infraestrutura	Quando o PR toca `infra/terraform/**`: `terraform fmt -check`, `validate` e `plan` (o `plan` é publicado para leitura).	`fmt`/`validate`: sim. `plan`: revisão humana
Build das imagens	`docker build` da imagem Lambda (`Dockerfile`) e das imagens self-hosted (`selfhosted/`), sem publicar.	Sim
Testes *golden* do SLM	`pytest -m golden` contra um Ollama real. Pesado demais para cada PR; roda no agendamento e sob demanda.	Não (alerta)

3.2 Quando uma versão está apta para implantação

Cada ambiente tem seu portão. Um código só passa para o ambiente seguinte quando **todas** as condições do portão forem verdadeiras:

Para entrar em	Condições	Quem aprova
DEV (deploy da branch)	CI verde no PR; branch atualizada com a `main`, sem conflitos; teste local feito.	Dev (dispara o deploy); Q.A aprova o resultado
Homologação (tag `vX.Y.Z-rc.N`)	Q.A aprovou no DEV; PR para a `main` revisado; branch de trabalho atualizada com a `main`; CI verde na release.	Tech Lead (publica a tag)
Produção (`main` + tag)	Q.A aprovou na homologação; migrações Alembic aplicam em banco vazio **e** sobre o esquema da versão em produção; `GET /health` respondendo; os 3 cenários do `dataspace-demo` (um conforme, um com erro heurístico, um com erro semântico) classificados como esperado; nenhum segredo no repositório; a `main` não difere do commit homologado além do merge; notas de release geradas.	Tech Lead (aprovação manual do deploy)

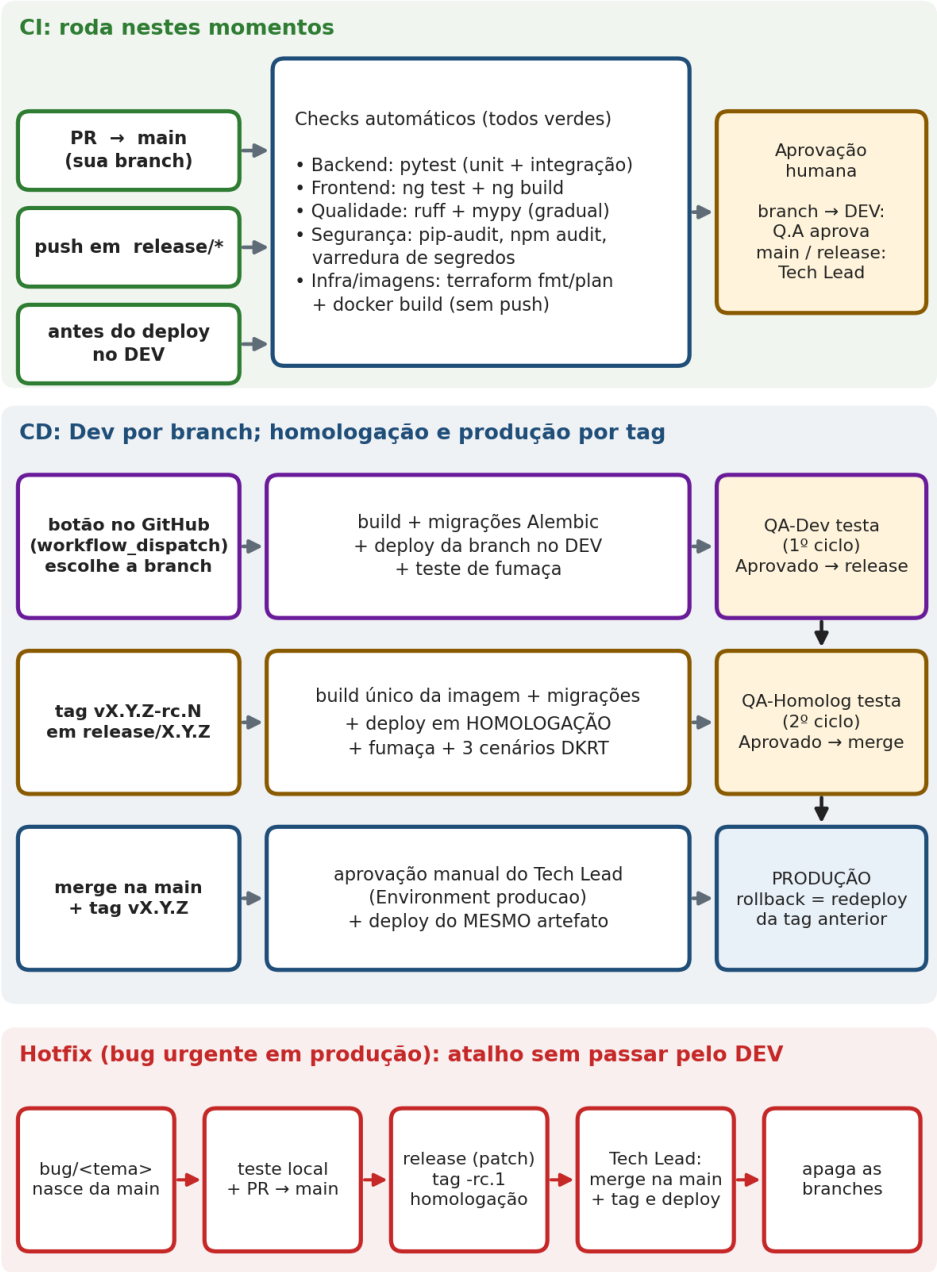


Figura 2: Pipeline de CI/CD por ambiente e fluxo de hotfix.

4. Entrega e implantação (CD)

4.1 Como o software chega a cada ambiente

O **DEV** recebe a branch de trabalho por deploy manual sob demanda (um workflow do GitHub Actions com o nome da branch como parâmetro). A **homologação e a produção são implantadas por tag**. Qualquer release pode ser homologada e levada à produção, sem uma branch permanente por ambiente: o Tech Lead cria uma `release/X.Y.Z` nova para cada alteração, publica nela a tag `vX.Y.Z-rc.N` (homologação) e, depois do merge, publica a tag `vX.Y.Z` na `main` (produção). Entre as duas vale o princípio **construir uma vez, promover o mesmo artefato**: a imagem Docker e o build do Angular gerados na tag `-rc` são os que sobem para produção, sem recompilar. Para isso ser verdade, a release precisa estar atualizada com a `main` antes de ser homologada, e o pipeline de produção confere que a `main` não difere do commit homologado; se a `main` andar durante a homologação, a release é atualizada e uma nova `-rc` é publicada. O artefato do DEV nunca é promovido: é gerado da branch e serve só para o primeiro ciclo de testes. O DEV é uma stack enxuta, definida em Terraform como os demais ambientes (`infra/terraform`).

Ambiente	Branch e gatilho	Automático	Aprovação humana
Local	Nenhuma. Manual, por quem desenvolve: `docker compose up -d` (Orion, Postgres), `uvicorn` e `ng serve`.	Não se aplica	Não
DEV	Nenhuma branch fixa: o Dev escolhe a branch no workflow de deploy (`workflow_dispatch`). Uma branch por vez (fila de *concurrency*).	Build, migrações, deploy e teste de fumaça, depois do clique	Q.A (1° ciclo), registrada no PR
Homologação (ADR-0018)	Tag `vX.Y.Z-rc.N` publicada na `release/X.Y.Z` pelo Tech Lead.	Build único da imagem, migrações, deploy, teste de fumaça e 3 cenários DKRT	Q.A (2° ciclo) e Tech Lead
Produção (ADR-0017)	Tag `vX.Y.Z` publicada na `main` pelo Tech Lead, após o merge da release.	Deploy do mesmo artefato após a aprovação	**Sim**, manual: Tech Lead
Self-hosted (ADR-0019)	Publicação da release.	Imagens versionadas publicadas no registry (GHCR)	Não. O produtor atualiza com `docker compose pull && up -d`

4.2 O que é automático e o que é manual

Automático	Manual (com responsável)
Build e testes Publicação da imagem e do build do frontend Migrações Alembic em DEV e homologação Deploy em homologação (tag `-rc`) Teste de fumaça (`/health`) e de aceitação (3 cenários) Publicação do frontend (S3 e invalidação do CloudFront) e deploy das funções Lambda em produção, após a aprovação Verificação de saúde após o deploy	**Testes do Q.A** nos ciclos DEV e homologação **Disparar o deploy da branch no DEV** (Dev) **Criar a `release/X.Y.Z` e publicar a tag `-rc.N`** (Tech Lead) **Merge na `main` e publicação da tag `vX.Y.Z`** (Tech Lead) **Aprovar a implantação em produção** (Tech Lead ou substituto) **`terraform apply`** (disparo manual, em ambiente com aprovação: `workflow_dispatch`) Publicação de release self-hosted para clientes

4.3 Quem autoriza uma implantação em produção

O ambiente `producao` do GitHub Environments exige a aprovação do **Tech Lead** (ou do substituto designado pelo grupo). A opção *Prevent self-review* impede que quem disparou a release aprove a própria implantação quando houver outro autorizador disponível. Nenhum outro papel consegue promover um artefato para produção, e o Q.A não faz deploy: seu papel é aprovar ou reprovar a versão.

4.4 Banco de dados, rollback e infraestrutura

- **Migrações:** o `alembic upgrade head` roda antes de o novo código receber tráfego. Como código antigo e novo podem coexistir por instantes, as migrações seguem a regra *expandir e contrair*: primeiro adicionam (colunas anuláveis, tabelas novas), e só uma versão depois removem o que ficou sem uso. Isso evita depender de *downgrade*.
- **Mais de uma branch com migração:** duas branches criadas da mesma revisão geram duas *heads* no Alembic. O CI falha nesse caso (`alembic heads`); a segunda branch a ser liberada traz a `main` para dentro de si e reaponta sua migração antes da homologação. O banco do DEV recebe uma branch por vez; quando a branch trocar, ele pode ser recriado a partir de um backup da produção anonimizado ou do esquema da `main`, para não carregar migrações de uma branch que não foi liberada.
- **Rollback:** reimplantar a tag anterior, já que os artefatos são imutáveis (versão e alias das Lambdas, build anterior do frontend com nova invalidação do CloudFront). Como as migrações são compatíveis, voltar o código não exige voltar o banco.
- **Infraestrutura (Terraform):** `plan` automático no PR, `apply` sempre manual. A infraestrutura muda raramente e um `apply` errado derruba o ambiente inteiro, então a aprovação humana vale o atrito.

5. Tratamento de bugs em produção

5.1 Quando usar o atalho de hotfix

O **atalho** é para falha que afeta a produção e **não pode esperar** o ciclo completo: sistema indisponível, auditoria retornando resultado errado, falha de segurança. A issue recebe o rótulo `urgente`, e o Tech Lead confirma o uso do atalho. Qualquer outro bug segue o fluxo completo da seção 2, com uma branch `bug/`.

5.2 Passo a passo

1. **Registrar:** abrir uma issue `urgente` com a severidade, o impacto e como reproduzir, e avisar o grupo e o Tech Lead.
2. **Criar a branch a partir da `main`:** `git switch -c bug/<descricao>`. Como a `main` é a produção, a correção parte exatamente do que está em uso, e nada de outras alterações em andamento vai junto.
3. **Corrigir com um teste que reproduz o bug**, escrito antes da correção. Mudança mínima, sem aproveitar para refatorar. Teste local completo.
4. **PR para a `main`** e envio da URL ao Tech Lead, com o CI completo. A revisão é obrigatória mesmo na urgência.
5. **Release de correção:** o Tech Lead cria `release/X.Y.Z` com o PATCH incrementado (de `1.2.0` para `1.2.1`), publica a tag `v1.2.1-rc.1` e a homologação roda o teste de fumaça, os 3 cenários do `dataspace-demo` e o caso que reproduz o bug. O Q.A aprova em um único ciclo, o da homologação.
6. **Publicar:** o Tech Lead faz o merge na `main`, publica a tag `vX.Y.Z` e aprova o deploy em produção.
7. **Incorporar nas demais branches:** branches em andamento recebem a correção quando trouxerem a `main` para dentro delas, o que é obrigatório antes do PR e antes da release.
8. **Encerrar:** apagar as branches, registrar na issue a causa raiz e o teste de regressão adicionado.

5.3 Se a correção demorar

Se a correção não for rápida, a primeira ação é **voltar a produção para a tag anterior** (seção 4.4), que restaura o serviço enquanto a branch `bug/` é preparada. O atalho elimina o primeiro ciclo (DEV), mas não elimina nem a revisão do Tech Lead nem o Q.A na homologação.

6. Funcionalidades ainda não concluídas

6.1 Decisão: a branch de trabalho é a unidade de entrega

Neste fluxo, uma funcionalidade só chega à produção quando a **release da sua branch** é aprovada. Enquanto está incompleta, ela vive na branch de trabalho (e, se for o caso, no DEV, para o Q.A testar) e **nunca chega à `main`**. Em outras palavras, a branch faz o papel de isolamento que, em outros processos, seria feito por uma feature flag. A **feature flag fica como exceção** para o que precisa entrar em produção em partes.

6.2 Justificativa

- **Coerência com o fluxo.** Como as branches nascem da `main` e a release sai da própria branch, uma funcionalidade pela metade não contamina a produção nem as demais releases. Isso é exatamente o que o desenho em funil protege.
- **O Q.A testa a funcionalidade inteira**, no DEV e na homologação. Uma flag dividiria o teste entre os estados ligado e desligado, o que dobra o trabalho de uma equipe pequena.
- **Custo proporcional.** Variáveis em `Settings` (como `DEPLOYMENT_MODE` e `QUEUE_BACKEND`) e o endpoint `/session`, que entrega configuração em runtime ao frontend (ADR-0020), dão à flag um custo quase nulo quando ela for necessária. Um serviço externo de flags seria desproporcional ao porte do projeto.
- **Risco conhecido e tratado.** Branches longas geram conflitos. As mitigações são a regra de 7 dias para branch parada (seção 1.4), a obrigação de trazer a `main` para dentro da branch antes do PR e da release, a divisão de trabalhos grandes em entregas menores e o primeiro ciclo de testes no DEV acontecer cedo.

6.3 Branch ou flag?

Situação	Estratégia
Funcionalidade que cabe em uma ou duas semanas e será entregue de uma vez	**Branch.** Segue o fluxo normal e só vira release quando aprovada.
Trabalho grande que pode ser dividido em partes independentes	**Várias branches**, cada uma com sua release, sem expor nada ao usuário até a última.
Parte que precisa entrar em produção antes do todo (ex.: migração *expandir*) ou funcionalidade a ser liberada em momento diferente do deploy	**Feature flag** `FEATURE_<NOME>`, desligada por padrão, ligada por ambiente em `Settings`/Parameter Store e exposta ao frontend em `/session`. Removida em até 2 releases depois de ligada.
Experimento descartável (*spike*)	**Branch própria** com vida de até 5 dias, apagada ao final, nunca mergeada.

7. Arquitetura da solução e pipelines

7.1 Mesma estratégia para tudo?

Sim. O projeto é um **monorepo**: backend, frontend, infraestrutura, bundle self-hosted e demonstração didática vivem no mesmo repositório, com uma só `main`, o mesmo fluxo de branches, o mesmo PR e o mesmo critério de aprovação. A arquitetura do código é um **monolito modular** (hexagonal), não microsserviços: há um único bounded context (auditoria de conformidade), um único banco e um único pacote Python que sobe como API ou como worker conforme o ponto de entrada.

Justificativa: numa equipe pequena, cada regra extra é custo de coordenação, e o contrato entre API e frontend muda junto (os DTOs do frontend espelham `schemas.py`), o que é mais simples de revisar e testar num mesmo PR, Dev e release.

7.2 Pipelines independentes por pasta

Mesma estratégia de branches, mas **pipelines independentes**, acionados por filtro de caminho (`paths:`). Assim cada mudança só executa o que lhe diz respeito. A **versão**, porém, é uma só: a release `X.Y.Z` cobre API e frontend, que são liberados juntos.

Componente	Pipeline	Dispara com mudança em	Artefato	Implantável isoladamente?
API + Worker	`backend`	`src/**`, `migrations/**`, `tests/**`, `requirements.txt`, `Dockerfile`	Imagem de contêiner (ECR)	Sim. As três funções (api, worker, worker-dlq) sobem juntas, por design, pois usam a mesma imagem (ADR-0017)
Frontend	`frontend`	`frontend/**`	Build estático (S3/CloudFront)	Sim, desde que o contrato da API seja compatível
Infraestrutura	`infra`	`infra/terraform/**`	Plano Terraform	Sim, com `apply` manual
Bundle self-hosted	`selfhosted`	`selfhosted/**` e as de backend/frontend	Imagens no GHCR	Sim, publicado só em release
dataspace-demo	`demo`	`dataspace-demo/**`	Nenhum (não é implantado)	Não se aplica. O CI roda `test_seed_dkrt.py`; seus cenários servem de massa de teste de aceitação

7.3 Versionamento e componentes implantados separadamente

- **Uma versão de produto** (`vX.Y.Z`) cobre API e frontend. A infraestrutura é rastreada por commit e `plan`, sem tag própria. O `dataspace-demo` é versionado junto, por ser didático.
- **Contrato API e frontend:** mudança incompatível é feita em duas etapas (expandir, depois contrair) ou vira versão MAJOR. Com isso o frontend pode ser publicado sem a API e vice-versa.
- **SLM (Ollama):** não tem build próprio; imagem pública e modelo definidos no Terraform. Trocar o modelo é mudança de infraestrutura, e os testes *golden* detectam regressão de qualidade.

7.4 Quando isso deveria mudar

A decisão não é definitiva: o grupo reavalia a separação em repositórios ou serviços quando dois componentes precisarem de ciclos de entrega diferentes. O candidato mais provável é o worker de auditoria, e a arquitetura hexagonal mantém essa extração barata.

8. Decisões e justificativas

Resumo das principais escolhas, com a alternativa descartada e o preço aceito. O enunciado pede coerência entre arquitetura, time, branches, integração e entrega: cada linha abaixo se apoia nas premissas da seção de contexto (equipe pequena, um produto, uma linha de produção, Q.A e Tech Lead como papéis).

Decisão	Alternativa descartada	Por que	Preço aceito
Fluxo em funil sem `develop`: `main` = produção; `release/*` e tags = homologação; DEV por deploy da própria branch	Git Flow com `develop` e branch `_develop`; GitHub Flow puro (só `main` e tags)	O time tem Q.A dedicado e quer dois ciclos de teste. Sem `develop`, o Q.A testa exatamente o que vira release, e some uma branch extra e um merge manual por alteração	O DEV atende uma branch por vez; alterações só se encontram ao trazer a `main` antes da homologação
Branches de trabalho nascem da `main`	Nascer de uma `develop` (Git Flow clássico)	Partindo da produção, a release leva só o que foi aprovado, sem código alheio	Conflitos com a `main` precisam ser resolvidos a cada alteração
DEV por deploy manual sob demanda (GitHub Actions)	Deploy automático de uma `develop`; ambiente temporário por PR	Simples e barato para um time pequeno; custo de infraestrutura baixo (um só ambiente)	Fila: um Dev por vez. Ambiente por PR é a evolução natural se a fila incomodar
Release nomeada (`release/X.Y.Z`) criada pelo Tech Lead; homologação e produção sobem por tag (`-rc.N` e `vX.Y.Z`)	Branch permanente `homologacao` ou deploy automático a cada push	Qualquer release pode ir a homologação e produção sem branch por ambiente. A tag é imutável e identifica exatamente o que foi testado; o deploy é uma decisão consciente do Tech Lead. Só o DEV, que muda o tempo todo, sobe por deploy manual	Várias tags `-rc` e uma homologação por vez, que pode virar gargalo; o Tech Lead precisa publicar as tags
Merge na `main` só pelo Tech Lead	Qualquer integrante mergeia após aprovação	Um responsável pela entrega e menos risco de merge sem revisão ou sem aprovação do Q.A	Gargalo no Tech Lead; mitigado pelo substituto designado
Merge commit da release na `main`	Squash merge	A release preserva o histórico dos commits e fica visível no log; a tag aponta para um commit nomeado	Histórico da `main` menos linear
Build único, artefato promovido da homologação para produção	Recompilar a cada ambiente	O que foi testado na homologação é o que vai para produção	Exige release atualizada com a `main` antes da homologação
Hotfix com atalho (sem passar pelo DEV)	Bug urgente seguindo o fluxo completo	Urgência: remove só o primeiro ciclo, mantendo revisão do Tech Lead e Q.A na homologação	Um ciclo de teste a menos; compensado pelo teste que reproduz o bug e pela homologação focada
Branch como unidade de entrega; flag como exceção	Flag para tudo, com a funcionalidade na `main` desligada	Coerente com o funil: o que não está aprovado não chega à `main`. O Q.A testa a funcionalidade inteira	Branches mais longas e risco de conflito (mitigado, seção 6.2)
Monorepo, pipelines por pasta	Multi-repositório ou microsserviços	Contrato API-frontend muda junto; um bounded context; equipe pequena	CI precisa de filtros de caminho bem mantidos

Referências

- Git Flow (Vincent Driessen): https://nvie.com/posts/a-successful-git-branching-model/
- GitHub Flow (alternativa considerada): https://docs.github.com/en/get-started/using-github/github-flow
- Conventional Commits: https://www.conventionalcommits.org
- Versionamento Semântico: https://semver.org/lang/pt-BR/
- Feature Toggles (Pete Hodgson): https://martinfowler.com/articles/feature-toggles.html
- HUMBLE, J.; FARLEY, D. Continuous Delivery. Addison-Wesley, 2010.
- Documentos do próprio projeto: ADR-0017 (produção em Lambda), ADR-0018 (homologação), ADR-0019 (modos SaaS e self-hosted), ADR-0020 (sessão e configuração em runtime), em `docs/adr/`.